

Design Pattern

Command

Adam HAMOUCH

May 2026

1. Intention

Encapsuler une action et ses parametres dans un **objet autonome**. Cela permet de :

- Planifier, mettre en file d'attente ou journaliser des actions.
- Implementer le **undo/redo** : chaque commande stocke l'etat necessaire pour s'annuler.
- Decoupler l'emetteur de la demande (Invoker) de l'executeur (Receiver).
- Traiter les actions comme des **donnees** : les serialiser, les envoyer sur le reseau, les rejouer.

2. Structure

- **Command** : interface avec `execute()` et `undo()`.
- **Commandes concretes** : implementent Command, stockent les parametres ET l'etat pour undo, deleguent le travail au Receiver.
- **Receiver** : l'objet qui fait vraiment le travail (mesh, scene, moteur physique...). La commande lui dit quoi faire.
- **Invoker** : stocke et declenche les commandes via `execute()`. Gere la queue et l'historique. Ne sait pas ce que fait la commande.
- **Client** : cree et configure les commandes, les donne a l'Invoker.

3. Exemple — Editeur de moteur (undo/redo)

```
// Interface Command
class ICommand {
public:
    virtual void execute() = 0;
    virtual void undo() = 0;
    virtual ~ICommand() = default;
};

// Receiver : l'objet qui fait le vrai travail
class SceneObject {
public:
    void setPosition(glm::vec3 pos) { position = pos; }
    glm::vec3 getPosition() const { return position; }
private:
    glm::vec3 position{0};
};

// Commande concrete : deplacer un objet
class MoveCommand : public ICommand {
public:
    MoveCommand(SceneObject& obj, glm::vec3 newPos)
        : object(obj)
        , oldPos(obj.getPosition()) // stocke l'etat pour undo
        , newPos(newPos) {}

    void execute() override { object.setPosition(newPos); }
    void undo() override { object.setPosition(oldPos); } // annulation gratuite
private:
    SceneObject& object;
    glm::vec3 oldPos;
    glm::vec3 newPos;
};

// Invoker : gere la queue et l'historique
```

```

class CommandHistory {
public:
    void execute(std::unique_ptr<ICommand> cmd) {
        cmd->execute();
        history.push(std::move(cmd)); // stocke pour undo
        while (!redoStack.empty()) redoStack.pop(); // invalide le redo
    }

    void undo() {
        if (history.empty()) return;
        auto cmd = std::move(history.top()); history.pop();
        cmd->undo();
        redoStack.push(std::move(cmd));
    }

    void redo() {
        if (redoStack.empty()) return;
        auto cmd = std::move(redoStack.top()); redoStack.pop();
        cmd->execute();
        history.push(std::move(cmd));
    }

private:
    std::stack<std::unique_ptr<ICommand>> history;
    std::stack<std::unique_ptr<ICommand>> redoStack;
};

// Client : cree et configure les commandes
CommandHistory editor;
SceneObject mesh;

editor.execute(std::make_unique<MoveCommand>(mesh, glm::vec3(1, 0, 0)));
editor.execute(std::make_unique<MoveCommand>(mesh, glm::vec3(2, 0, 0)));
editor.undo(); // mesh revient a (1, 0, 0)
editor.redo(); // mesh revient a (2, 0, 0)

```

4. Cas d'usage engine dev

- **Editeur** : chaque action utilisateur est une Command → undo/redo gratuit.
- **Render thread** : le game thread pousse des RenderCommand dans une queue, le render thread les consomme. C'est exactement ce que fait Unreal avec ENQUEUE_RENDER_COMMAND.
- **Networking** : les commandes sont serializables → envoyer sur le reseau, rejouer sur un autre client.
- **Replay system** : enregistrer toutes les commandes d'une session → rejouer la partie exactement.

Unreal : ENQUEUE_RENDER_COMMAND pousse une lambda comme commande dans la queue du render thread. Le game thread ne touche jamais directement au render state — il envoie des commandes.

```

ENQUEUE_RENDER_COMMAND(UpdateTransform)(
    [mesh, newTransform](FRHICommandList& RHICmdList) {
        mesh->SetTransform(newTransform); // execute sur le render thread
    }
);

```

5. Avantages / Quand utiliser

Avantages :

- Undo/redo : chaque commande stocke ce qu'il faut pour s'annuler.
- Decouplage total entre l'emetteur et le recepteur.
- Les commandes sont des donnees : file d'attente, serialisation, replay.
- Open/Closed : ajouter une action = ajouter une classe, rien a modifier.

Utiliser si :

- Undo/redo necessaire (editeur, outils).
- Communication differee entre threads (render command queue).
- Actions a journaliser ou rejouer (networking, replay).

Eviter si : action simple sans besoin d'annulation ni de file — appel direct suffit.